Metasploit Integration — Added Files

Cerberus backend · pentest framework external-tool layer

commit 8fd3935 · 12 files · +1031 lines

What was added

A new **external-tool integration layer** inside the pentest framework, with Metasploit as its first adapter. The layer is deliberately **tool-agnostic**: the framework talks to a `ToolAdapter` interface, so the next tool (nmap, sqlmap, nuclei, …) plugs in exactly the same way — implement the ABC, drop it in `pentest/tools/`, done.

Two layers were extended:

- **`pentest/tools/`** — the tool adapters themselves (interface, registry, Metasploit implementation).
- **`pentest/modules/tools/`** — the attack-framework module that consumes the adapter, with the safety gates.

File	Status	Lines	Purpose
`pentest/tools/base_tool.py`	NEW	74	The tool-agnostic contract: `ToolAdapter` ABC (`connect` , `disconnect` , `is_connected` , `ensure_connected` , `execute` , `get_metadata`), `ToolStatus` enum, `enabled_by_default = False` opt-in flag. Every future tool (nmap, sqlmap, nuclei…) implements this one interface.
`pentest/tools/metasploit_adapter.py`	NEW	214	The Metasploit adapter. Speaks the `msfrpcd` XML-RPC API using ONLY the Python standard library (`xmlrpc.client`) — zero new dependencies. All blocking calls wrapped in `asyncio.to_thread` . Console-based module execution (create → write → read → destroy), so it works without a Metasploit database. Timeout + self-signed-TLS support. Defense-in-depth: refuses `exploit/*` unless the caller passes an RoE-derived `allow_exploits` flag.
`pentest/tools/tool_registry.py`	NEW	98	`ToolRegistry` with package auto-discovery — the same pattern as the framework's `ModuleRegistry` . Dropping a `ToolAdapter` subclass into `pentest/tools/` registers it automatically; no boilerplate anywhere else.
`pentest/tools/__init__.py`	NEW	23	Package exports for the tool layer.
`pentest/modules/tools/__init__.py`	NEW	2	Package marker for the new `tools` module category.
`pentest/modules/tools/metasploit_scanner.py`	NEW	239	The attack-framework module (`metasploit_scanner` , category `tools` , MITRE T1046/T1087, impact = proof-of-concept). CODE-enforced module-type gate: auxiliary-only; `exploit/*` refused unless the Rules of Engagement grant FULL impact. Sets RHOSTS/THREADS from the engagement context, parses `host:port - TCP OPEN` console lines into per-port evidence, and fails soft (INFO finding) when the tool is disabled or unreachable.
`pentest/core/exceptions.py`	MODIFIED	+6	Added `ToolUnavailableError` — the fail-soft signal the engine and modules use to degrade gracefully when a tool daemon is down.
`app/core/config.py`	MODIFIED	+12	New `MSF_*` settings block: `MSF_ENABLED` (default `false`), `MSF_RPC_HOST` (127.0.0.1), `MSF_RPC_PORT` (55553), `MSF_RPC_PASSWORD` , `MSF_RPC_SSL` (false), `MSF_RPC_TIMEOUT` (10s).
`pentest/__init__.py`	MODIFIED	+14	Exports the tool layer (`MetasploitAdapter` , `ToolAdapter` , `ToolRegistry` , `ToolStatus` , `default_tool_registry`) alongside the existing framework API.
`backend/.env.example`	MODIFIED	+10	Documents the five `MSF_*` variables and the one-line `msfrpcd` startup command.

`tests/unit/test_pentest_tools.py`	NEW	320	15 tests, including a real in-process fake `msfrpcd` (stdlib XML-RPC server) so the adapter's actual protocol path is exercised: auth, console create/write/read/destroy, module listing, exploit refusal, RoE escalation, port parsing, and the fail-soft contract.
`scripts/live_tests/pentest_framework_smoke.py`	MODIFIED	+19	Live smoke now also reports the tool registry and runs the `metasploit_scanner` module (expect INFO when disabled — the fail-soft contract).

How the pieces connect

```
pentest/modules/tools/metasploit_scanner.py   (attack module, category "tools")
        │  safety gate: auxiliary-only, exploits need FULL-impact RoE
        ▼
pentest/tools/metasploit_adapter.py           (ToolAdapter implementation)
        │  XML-RPC over HTTP(S), stdlib only, asyncio.to_thread
        ▼
msfrpcd  ──→  Metasploit Framework (consoles; no MSF database required)
```

Safety invariants (all code-enforced)

1. **Opt-in** — `MSF_ENABLED=false` by default; disabled state yields an INFO finding, never an error.
2. **Fail-soft** — unreachable `msfrpcd` → INFO finding; an engagement never crashes because a tool is down.
3. **Scope first** — the framework's `SafetyGuard` checks targets against the engagement scope *before* any module code runs.
4. **Auxiliary-only** — `exploit/*` modules are refused by the module *and* the adapter unless the RoE grants FULL impact.

Verified: 76/76 unit tests, ruff/black/mypy clean, live smoke exercised, API healthy.